



**KIET**  
GROUP OF INSTITUTIONS  
*Connecting Life with Learning*

**A**  
**Project Synopsis**  
on  
**SecurAI Analyzer: A Hybrid Rule-and-Learning Platform for  
Source Code Vulnerability Detection and Deepfake Detection**

submitted as partial fulfillment for the award of  
**BACHELOR OF TECHNOLOGY  
DEGREE**

SESSION 2026-27  
in  
**Computer Science and Engineering**

By  
Rachit Agarwal (Roll No. 2300290100194) (Team Leader)  
Raunak Kushwaha (Roll No. 2300290100203)  
Pravira Shukla (Roll No. 2300290100183)  
Raunak Kumar Bharti (Roll No. 2300290100202)

**Under the supervision of**  
Ms. Deepti Singh  
**KIET Group of Institutions, Ghaziabad**

Affiliated to  
**Dr. A.P.J. Abdul Kalam Technical University, Lucknow**  
(Formerly UPTU)  
**Academic Year 2026-27**

## **Abstract**

Modern organisations face two security threats that are usually handled by completely separate tools: vulnerabilities hidden in their own source code, and deepfake images used for fraud and impersonation. SecurAI Analyzer is a single platform that addresses both, using the same underlying idea for each: combine a set of hand-written detection rules with a machine learning model, and let the two check each other rather than trusting either one blindly.

For code, the system runs 37 static-analysis rules covering common weaknesses such as SQL injection, command injection, path traversal, insecure deserialization, and hardcoded secrets, and pairs this with a Random Forest classifier trained to flag suspicious patterns a rule might miss. For images, the system uses Community Forensics, a publicly available Vision Transformer trained on 2.7 million images from 4,803 different generators, combined with six classical forensic checks such as error-level analysis and metadata inspection.

What makes this project different from simply wiring two tools together is that every design decision was tested rather than assumed. During development, the team discovered that the machine-learning components in both halves of the system, if trusted too much, actually hurt accuracy instead of helping it: a leakage issue in the training data made an early vulnerability classifier look better than it really was, and combining forensic signals with the deepfake model lowered its accuracy instead of raising it. Both issues were found, measured, and corrected before the final system was built, and the resulting design keeps machine learning outputs visible as supporting evidence rather than as the final word.

## **1. Introduction**

Software teams and the general public are increasingly targeted by two very different kinds of attacks that nonetheless share a common weakness: both rely on something looking legitimate when it is not. On the software side, a single missed vulnerability in application code, such as an unsanitised database query or a hardcoded password, can let an attacker into a system that otherwise looks secure. On the media side, a convincingly generated deepfake image can be used to impersonate a real person for fraud, harassment, or misinformation, and the generators producing these images are improving every year.

Existing tools tend to treat these as two unrelated problems solved by two unrelated kinds of software: static analysis scanners for code, and deep-learning classifiers for images. Both

categories of tool are also usually built the same questionable way: a rule-based or machine-learning component is added to a system because it seems like it should help, without anyone actually measuring whether it does. SecurAI Analyzer was built to test that assumption directly, applying one consistent method — combine a deterministic, explainable component with a learned model, and measure the learned model's real contribution before trusting it — to both problems within a single platform.

The project is built as three independent services: a React-based web client, a Node.js/Express backend that runs the 37-rule vulnerability scanner and computes an overall risk score, and two Python services running the machine learning components — a lightweight classifier for code, and the deepfake detection pipeline. The deepfake service is kept separate from the rest of the system because it needs heavier dependencies and takes noticeably longer to respond, and isolating it keeps the fast code-scanning path from slowing down.

During the course of building and testing the system, the team found and fixed three concrete problems that are usually invisible in student and even published projects: a data-leakage issue in the vulnerability classifier's training data that made its reported accuracy meaningless, a case where adding a second detection signal to the deepfake model made results worse rather than better, and a mismatch between how the deepfake model was tested and how it actually behaved once deployed. Each of these is documented in this project not as a flaw to hide but as evidence that the system's components were genuinely tested rather than assumed to work.

## **2. Literature Survey**

Vulnerability detection research has moved steadily from purely rule-based static analysis tools towards hybrid systems that add a machine-learning layer on top. Shiri Harzevili et al. [1] and Shaon and Akter [2] both survey this shift and note that most hybrid tools still lean heavily on rule-based features even when a learned model is present, meaning the model rarely contributes much reasoning of its own. Mweu and Ndia [3] reviewed static-analysis literature between 2017 and 2025 and found that hybrid methods are rarely evaluated on properly held-out test data, which makes published accuracy numbers difficult to trust at face value. Orji et al. [4] combined static analysis with a large language model for PHP vulnerability detection, but did not check whether their test cases overlapped with their model's training prompts, which is exactly the kind of leakage risk this project encountered and had to fix.

On the deepfake detection side, An, Wang, and Joe [5] compared a CNN-based classifier against a Random Forest baseline and found the CNN performed somewhat better, though both were tested only on synthetic data. Park and Owens [6] introduced Community Forensics, the pretrained detector used in this project, and showed that training on a very large and diverse set of generators improves real-world generalisation far more than changing the model architecture does. Nadimpalli and Rattani [7] highlighted a problem this project explicitly checked for: deepfake detectors that score well on the dataset they were trained on can drop sharply when tested on a different one, which is why this project's results are reported with that limitation clearly stated rather than implied to generalise further than they do.

### **3. Methodology / Planning of Work**

The project was carried out in the following stages:

1. Requirement study: identifying the vulnerability categories (OWASP Top 10, common CWEs) and deepfake threat types the system needed to address.
2. Rule engine development: writing and testing 37 static-analysis rules covering injection, unsafe deserialization, path traversal, secret leakage, weak cryptography, and related weaknesses, each mapped to a CWE/OWASP category.
3. Machine learning component development: training a Random Forest classifier for code, and integrating the pretrained Community Forensics Vision Transformer for image analysis, along with six supporting forensic checks.
4. System integration: building the React client, Node.js/Express backend, and two Flask microservices, and connecting them into a single working platform.
5. Testing and validation: building an independent held-out test set separate from the data used during development, and measuring the actual contribution of each machine-learning component rather than assuming it helped.
6. Issue diagnosis and correction: identifying and fixing the data-leakage, fusion-weight, and benchmark-mismatch issues discovered during testing.
7. Documentation and reporting: preparing the project report and an accompanying research paper describing the system and its evaluation.

### **4. Facilities Required for Proposed Work**

**Software:**

- Node.js and npm (backend and frontend development)
- Python 3.10+ with scikit-learn, PyTorch, and OpenCV (machine learning components)
- React.js (frontend client)
- MongoDB (scan history storage)
- Visual Studio Code / any standard IDE
- Git and GitHub (version control)

**Hardware:**

- A standard development laptop/desktop (minimum 8 GB RAM recommended)
- Internet access for downloading the pretrained Community Forensics model from Hugging Face

**References**

- [1] N. Shiri Harzevili, A. Boaye Belle, J. Wang, S. Wang, Z. M. Jiang, and N. Nagappan, “A Systematic Literature Review on Automated Software Vulnerability Detection Using Machine Learning,” *ACM Computing Surveys*, vol. 57, no. 3, art. 55, 2024.
- [2] M. S. H. Shaon and M. S. Akter, “Modern Approaches to Software Vulnerability Detection: A Survey of Machine Learning, Deep Learning, and Large Language Models,” *Electronics (MDPI)*, vol. 14, no. 22, art. 4449, 2025.
- [3] B. Mweu and J. Ndia, “Static Analysis Techniques for Secure Software: A Systematic Review,” *Journal of Cyber Security*, vol. 7, no. 1, pp. 417–437, 2025.
- [4] C. E. Orji, C. C. Ononiwu, and T. N. Ukachukwu, “Static Analysis Meets Artificial Intelligence: A Hybrid Framework for PHP Vulnerability Detection Using DeepSeek LLM and Machine Learning,” *Int. J. Latest Technol. Eng. Manag. Appl. Sci.*, vol. 15, no. 6, 2026.
- [5] J. H. An, Z. Wang, and I. Joe, “A CNN-based automatic vulnerability detection,” *J. Wireless Com. Network*, vol. 2023, art. 41, 2023.
- [6] J. Park and A. Owens, “Community Forensics: Using Thousands of Generators to Train Fake Image Detectors,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2025.

[7] A. V. Nadimpalli and A. Rattani, "On Improving Cross-dataset Generalization of Deepfake Detectors," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition Workshops (CVPRW), 2022.